CSE141 Major Task — Operations Guide

Expected Behavior & Sample Outputs — Spring 2026

Project 1: Bank Queuing System

Main Menu

Upon running the program, display the following menu:

```
╔═══════════════════════════════════╗
║   BANK CUSTOMERS QUEUING SYSTEM   ║
╚═══════════════════════════════════╝


1. System Configuration
2. Run Simulation
3. View Reports
4. View Statistics
5. Exit

Enter your choice: _
```

The user selects an operation by entering its number. If they enter an invalid value, display an error and re-prompt.

Operation 1: System Configuration

Upon choosing this option, display the submenu:

```
=== System Configuration ===
1. Set Number of Tellers
2. Set Working Hours
3. Define Transaction Types
4. Set Yearly Customer Target
5. View Current Configuration
6. Back to Main Menu

Enter your choice: _
```

1.1 Set Number of Tellers

The system should ask the user to enter the number of tellers for each working day.

Example interaction:

```
Enter number of tellers for Sunday: 3
Enter number of tellers for Monday: 4
Enter number of tellers for Tuesday: 4
...
```

Expected error checks:

- Number of tellers must be a positive integer (≥ 1)
- Number of tellers should not exceed MAX_TELLERS (default: 10)

1.2 Set Working Hours

The system should ask for the opening and closing times.

Example interaction:

```
Enter opening time (HH:MM): 09:00
Enter closing time (HH:MM): 15:00
```

Expected error checks:

- Time must be in valid HH:MM format
- Opening time must be before closing time
- Hours: 00-23, minutes: 00-59

1.3 Define Transaction Types

The system should allow adding transaction types with their average durations.

Example interaction:

```
Enter transaction name (or 'done' to finish): Deposit
Enter average duration in minutes: 3
Transaction added successfully!

Enter transaction name (or 'done' to finish): Withdrawal
Enter average duration in minutes: 5
Transaction added successfully!

Enter transaction name (or 'done' to finish): done
```

Expected error checks:

- Transaction name must not be empty
- Duration must be a positive integer
- Transaction name must be unique

1.4 Set Yearly Customer Target

Example interaction:

```
Enter target number of customers per year: 36000
Target set successfully! (Average ~100 customers/day)
```

Expected error checks:

- Value must be an integer greater than 30,000

Operation 2: Run Simulation

Upon choosing this option, display the submenu:

```
=== Run Simulation ===
1. Run Daily Simulation
2. Run Weekly Simulation
3. Run Full Year Simulation
4. Back to Main Menu

Enter your choice: _
```

2.1 Run Daily Simulation

The system should simulate one day of bank operations by:

1. Generating random number of customers (based on yearly target average)
2. Assigning random arrival times within working hours
3. Assigning random transaction types to each customer
4. Processing customers in first-come-first-served order
5. Tracking waiting times and teller utilization

Example output:

```
Simulating Sunday...
Customers Generated: 98
Processing customers...
[======================================] 100%

=== Daily Summary ===
Customers Served: 95
Customers Not Served: 3
Average Waiting Time: 8.5 minutes
Peak Hour: 11:00 AM - 12:00 PM (23 customers)
```

Possible outcomes:

- **Configuration Missing:** If tellers or transaction types not configured
- **Simulation Complete:** When the simulation finishes successfully

---

Operation 3: View Reports

Upon choosing this option, display the submenu:

```
=== View Reports ===
1. View Customer Report
2. View Teller Report
3. Export Reports to Files
4. Back to Main Menu

Enter your choice: _
```

3.1 View Customer Report

The system should display all customer records from the last simulation.

Example output:

ID	Arrival	Transaction	Svc Start	Svc End	Served
001	09:05	Deposit	09:05	09:08	Yes
002	09:08	Withdrawal	09:08	09:13	Yes
003	09:10	Loan Payment	09:13	09:23	Yes

3.2 View Teller Report

Example output:

Teller ID	Idle Time	Working Time	Utilization	Customers
T001	45 min	315 min	87.5%	42
T002	60 min	300 min	83.3%	38

3.3 Export Reports to Files

The system should save reports to `customers_report.txt` and `tellers_report.txt`.

Example interaction:

```
Exporting customer report... Done!
Exporting teller report... Done!
Reports saved successfully!
```

Operation 4: View Statistics

The system should display computed statistics from all simulations:

Example output:

```
=== Simulation Statistics ===
Total Simulation Days: 5
Total Customers: 487
Average Waiting Time: 7.3 minutes
Peak Hour (Overall): 11:00 AM - 12:00 PM
Busiest Day: Monday (112 customers)
Average Teller Utilization: 82.4%
```

Project 2: Student Information System

Main Menu

Upon running the program, display the following menu:

```
╔═══════════════════════════════════════╗
║     STUDENT INFORMATION SYSTEM        ║
╚═══════════════════════════════════════╝


1. Student Management
2. Course Management
3. Grades Management
4. Exit

Enter your choice: _
```

The user selects an operation by entering its number. If they enter an invalid value, display an error and re-prompt.

Operation 1: Student Management

Upon choosing this option, display the submenu:

```
=== Student Management ===
1. Add New Student
2. Search Student
3. Update Student
4. Delete Student
5. List All Students
6. Back to Main Menu

Enter your choice: _
```

1.1 Add New Student

The system should ask for all student fields, validate each one, and display appropriate error messages for invalid inputs.

Example interaction:

```
Enter Student Name: Ahmed Mohamed
Enter National ID: 30201011234567
Enter Gender (M/F): M
Enter Date of Birth (DD/MM/YYYY): 15/03/2005
Enter Phone Number: 01012345678
Enter Program (CSE/CCE/MCT): CSE
Enter Academic Level (1-4): 1

Student added successfully!
Student ID: 26P0001
```

Expected error checks:

- **Name:** Must be exactly two words (first and last name)
- **National ID:** Must be exactly 14 digits, no leading zero, must be UNIQUE
- **Gender:** Must be 'M' or 'F' (case insensitive)
- **Date of Birth:** Must be valid DD/MM/YYYY format, age ≥ 17
- **Phone Number:** Must be 11 digits starting with "01"
- **Program:** Must be CSE, CCE, or MCT
- **Academic Level:** Must be 1, 2, 3, or 4

Possible outcomes:

- **Invalid Input:** Display specific error message and re-prompt
- **Duplicate National ID:** "ERROR: A student with this National ID already exists"
- **Operation Successful:** Display generated Student ID

1.2 Search Student

Upon choosing this option, ask the user how they want to search:

```
=== Search Student ===
1. Search by Student ID
2. Search by Name
3. Search by National ID
4. Back

Enter your choice: _
```

Note: Student ID and National ID are unique (max 1 result). Name may return multiple students.

Example interaction (search by name):

```
Enter student name to search: Ahmed Mohamed

Found 2 student(s):

╔══════════════════════════════════════════════╗
║ Student ID: 26P0001                          ║
║ Name: Ahmed Mohamed                          ║
║ National ID: 30201011234567                  ║
║ Gender: Male | DOB: 15/03/2005               ║
║ Phone: 01012345678                           ║
║ Program: CSE | Level: 1 | GPA: 3.25          ║
╚══════════════════════════════════════════════╝

╔══════════════════════════════════════════════╗
║ Student ID: 2600015                          ║
║ Name: Ahmed Mohamed                          ║
║ National ID: 30305221234567                  ║
║ Gender: Male | DOB: 22/05/2003               ║
║ Phone: 01198765432                           ║
║ Program: CCE | Level: 2 | GPA: 2.85          ║
╚══════════════════════════════════════════════╝
```

Possible outcomes:

- **Invalid Search Value:** Display error for invalid format
- **Student Not Found:** "No student found with the given criteria"
- **Operation Successful:** Display student record(s)

1.3 Update Student

The system should ask for the Student ID, then show which fields can be updated:

```
Enter Student ID to update: 26P0001

Student found: Ahmed Mohamed

Which field do you want to update?
1. Phone Number
2. Program
3. Academic Level
4. Cancel

Enter your choice: _
```

Possible outcomes:

- **Invalid ID:** "ERROR: Invalid Student ID format"
- **Student Not Found:** "No student exists with ID: 26P0099"
- **Invalid New Value:** Display specific validation error
- **Operation Successful:** "Student record updated successfully"

1.4 Delete Student

The system should ask for Student ID and confirm before deletion:

Example interaction:

```
Enter Student ID to delete: 26P0001

Student found: Ahmed Mohamed
Are you sure you want to delete this student? (Y/N): Y

Student deleted successfully!
```

Possible outcomes:

- **Invalid ID:** "ERROR: Invalid Student ID format"
- **ID Not Found:** "No student exists with ID: 26P0099"
- **Operation Cancelled:** User entered 'N'
- **Operation Successful:** "Student deleted successfully"

1.5 List All Students

The system should ask for sort order:

```
=== List All Students ===
Sort by:
1. Student ID
2. Name (A-Z)
3. GPA (Highest First)
4. Back

Enter your choice: _
```

Example output:

```
Total Students: 25

╔═══════════╦═══════════════════════╦═══════════╦═══════╦═══════╗
║ ID        ║ Name                  ║ Program   ║ Level ║ GPA   ║
╠═══════════╬═══════════════════════╬═══════════╬═══════╬═══════╣
║ 26P0001   ║ Ahmed Mohamed         ║ CSE       ║ 1     ║ 3.25  ║
║ 26P0002   ║ Sara Ahmed            ║ CCE       ║ 2     ║ 3.80  ║
║ 2600003   ║ Omar Hassan           ║ MCT       ║ 1     ║ 2.95  ║
╚═══════════╩═══════════════════════╩═══════════╩═══════╩═══════╝
```

Operation 2: Course Management

Upon choosing this option, display the submenu:

```
=== Course Management ===
1. Add New Course
2. View All Courses
3. Update Course
4. Delete Course
5. Back to Main Menu

Enter your choice: _
```

2.1 Add New Course

Example interaction:

```
Enter Course Code: CSE141
Enter Course Title: Introduction to Programming
Enter Credit Hours (1-4): 3

Course added successfully!
```

Expected error checks:

- **Course Code:** Must be 3 letters + 3 digits (Ex: CSE141), must be UNIQUE
- **Course Title:** Must not be empty
- **Credit Hours:** Must be 1, 2, 3, or 4

Operation 3: Grades Management

Upon choosing this option, display the submenu:

```
=== Grades Management ===
1. Enter Student Grades
2. View Student Grades
3. Calculate GPA
4. Generate Transcript
5. Back to Main Menu

Enter your choice: _
```

3.1 Enter Student Grades

Example interaction:

```
Enter Student ID: 26P0001
Enter Course Code: CSE141

Enter Midterm Grade (0-40): 35
Enter Final Grade (0-60): 50

Total: 85/100
Letter Grade: A
Grade recorded successfully!
```

Expected error checks:

- **Student ID:** Must exist in the system
- **Course Code:** Must exist in the system
- **Midterm Grade:** Must be 0-40
- **Final Grade:** Must be 0-60

3.4 Generate Transcript

Example interaction:

```
Enter Student ID: 26P0001
```

```
╔═══════════════════════════════════════════════════════════╗
║                     STUDENT TRANSCRIPT                    ║
╠═══════════════════════════════════════════════════════════╣
║ Student ID: 26P0001                                       ║
║ Name: Ahmed Mohamed                                       ║
║ Program: CSE | Level: 1                                   ║
╠═══════════╦═══════════════════════╦═══════╦═══════╦═══════╣
║ Course    ║ Title                 ║ Credits ║ Grade ║Pts  ║
╠═══════════╬═══════════════════════╬═══════╬═══════╬═══════╣
║ CSE141    ║ Intro to Programming  ║   3   ║   A   ║4.0  ║
║ MTH101    ║ Calculus I            ║   4   ║   B   ║3.0  ║
║ PHY101    ║ Physics I             ║   3   ║   A   ║4.0  ║
╠═══════════╩═══════════════════════╩═══════╩═══════╩═══════╣
║ Total Credit Hours: 10                                    ║
║ Cumulative GPA: 3.60                                      ║
╚═══════════════════════════════════════════════════════════╝
```

Project 3: Study Schedule Generator

Main Menu

Upon running the program, display the following menu:

```
╔═══════════════════════════════════════╗
║     STUDY SCHEDULE GENERATOR          ║
╚═══════════════════════════════════════╝


1. Room Management
2. Subject Management
3. Section Management
4. Instructor Preferences
5. Generate Schedule
6. View Schedule
7. Exit

Enter your choice: _
```

Operation 1: Room Management

Upon choosing this option, display the submenu:

```
=== Room Management ===
1. Add Room
2. View All Rooms
3. Update Room
4. Delete Room
5. Back to Main Menu

Enter your choice: _
```

1.1 Add Room

Example interaction:

```
Enter Room ID: L01
Enter Room Name: Computer Lab 1
Enter Room Type (1=Lecture Hall, 2=Lab, 3=Tutorial Room): 2
Enter Capacity: 30

Room added successfully!
```

Expected error checks:

- **Room ID:** Must be 1 letter + 2 digits, must be UNIQUE
- **Room Name:** Must not be empty
- **Room Type:** Must be 1, 2, or 3
- **Capacity:** Must be a positive integer

Operation 2: Subject Management

Upon choosing this option, display the submenu:

```
=== Subject Management ===
1. Add Subject
2. View All Subjects
3. Update Subject
4. Delete Subject
5. Back to Main Menu

Enter your choice: _
```

2.1 Add Subject

Example interaction:

```
Enter Subject Code: CSE141
Enter Subject Name: Introduction to Programming
Enter Lecture Hours per week (0-4): 2
Enter Tutorial Hours per week (0-2): 1
Enter Lab Hours per week (0-4): 2
Enter Instructor Name: Dr. Ahmed Hassan

Subject added successfully!
```

Expected error checks:

- **Subject Code:** Must be 3 letters + 3 digits, must be UNIQUE
- **Lecture Hours:** Must be 0-4
- **Tutorial Hours:** Must be 0-2
- **Lab Hours:** Must be 0-4
- **Total Hours:** Sum must be greater than 0

Operation 3: Section Management

Upon choosing this option, display the submenu:

```
=== Section Management ===
1. Add Section
2. View All Sections
3. Update Section
4. Delete Section
5. Back to Main Menu

Enter your choice: _
```

3.1 Add Section

Example interaction:

```
Select Program:
1. CSE    2. CCE    3. MCT

Enter choice: 1
Select Level (1-4): 1
Enter number of sections: 3
Sections added: CSE Level 1 - 3 sections (A, B, C)
```

Expected error checks:

- **Program:** Must be 1, 2, or 3
- **Level:** Must be 1-4
- **Section Count:** Must be a positive integer

Operation 4: Instructor Preferences

Upon choosing this option, display the submenu:

```
=== Instructor Preferences ===
1. Set Unavailable Time Slots
2. View Instructor Preferences
3. Clear Preferences
4. Back to Main Menu

Enter your choice: _
```

4.1 Set Unavailable Time Slots

Example interaction:

```
Enter Instructor Name: Dr. Ahmed Hassan

Current unavailable slots: None

Select day:
1. Sunday    2. Monday    3. Tuesday
4. Wednesday 5. Thursday

Enter day number: 5

Enter time slot (1-6):
  1. 08:00-09:00    2. 09:00-10:00    3. 10:00-11:00
  4. 11:00-12:00    5. 12:00-01:00    6. 01:00-02:00

Enter slot number: 1

Dr. Ahmed Hassan marked as unavailable on Thursday 08:00-09:00
Add another? (Y/N): N
```

Operation 5: Generate Schedule

Upon choosing this option, the system should validate all data and generate a conflict-free schedule.

Example interaction:

```
Validating data...
✓ 10 rooms found
✓ 15 subjects found
✓ 12 sections found
✓ 8 instructors found
Generating schedule...
[======================================] 100%
Schedule generated successfully!
Total sessions scheduled: 156
Conflicts resolved: 0

Would you like to view the schedule? (Y/N): _
```

Schedule Generation Rules

The system must enforce these rules:

6. **No Instructor Conflict:** An instructor cannot teach two sessions simultaneously
- *Error:* "CONFLICT: Dr. Ahmed assigned to CSE141 and MTH101 at Sunday 10:00"
7. **No Room Conflict:** A room cannot be double-booked
- *Error:* "CONFLICT: Room L01 assigned to CSE141 and CSE241 at Monday 09:00"
8. **No Student Conflict:** A student group cannot have overlapping sessions
- *Error:* "CONFLICT: CSE Level 1 Section A has PHY101 and MTH101 at Tuesday 11:00"
9. **Room Type Match:** Labs must be scheduled in lab rooms only
- *Error:* "INVALID: CSE141 Lab cannot be scheduled in Lecture Hall A"
10. **Instructor Availability:** Respect unavailable time slots
- *Error:* "INVALID: Dr. Ahmed is unavailable on Thursday 08:00-09:00"
11. **Gap Minimization:** Try to have no more than 2 free slots per day per group

---

Operation 6: View Schedule

Upon choosing this option, display the submenu:

```
=== View Schedule ===
1. View by Program/Level
2. View by Instructor
3. View by Room
4. Export to File
5. Back to Main Menu

Enter your choice: _
```

6.1 View by Program/Level

Example interaction:

```
Select Program:
1. CSE    2. CCE    3. MCT

Enter choice: 1

Select Level (1-4): 1

Select Section:
1. Section A    2. Section B    3. Section C

Enter choice: 1
```

CSE - LEVEL 1 - SECTION A

Time	Sunday	Monday	Tuesday	Wednesday
08:00-09:00	CSE141 Lec Hall A	MTH101 Lec Hall B	PHY101 Lec Hall A	ENG101 Hall C
09:00-10:00	CSE141 Lec Hall A	MTH101 Tut T01	PHY101 Tut T02	---
10:00-11:00	CSE141 Lab Lab 1	---	---	MTH101 Lec Hall B